

Цель Блокнота

Рассмотрение механизмов интерпритации моделей машинного обучения в рамках библиотеки `shap`

Import библиотек

Обновим `scikit-learn` чтобы воспользоваться некими новинками

Например `TargetEncoder`

```
!pip install -U scikit-learn
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.11/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (3.5.0)
```

```
!pip install catboost
```

```
Collecting catboost
  Downloading catboost-1.2.7-cp311-cp311-manylinux2014_x86_64.whl.metadata (1.2 kB)
Requirement already satisfied: graphviz in /usr/local/lib/python3.11/dist-packages (from catboost) (0.20.3)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (from catboost) (3.10.0)
Requirement already satisfied: numpy<2.0,>=1.16.0 in /usr/local/lib/python3.11/dist-packages (from catboost) (1.26.4)
Requirement already satisfied: pandas>=0.24 in /usr/local/lib/python3.11/dist-packages (from catboost) (2.2.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from catboost) (1.13.1)
Requirement already satisfied: plotly in /usr/local/lib/python3.11/dist-packages (from catboost) (5.24.1)
Requirement already satisfied: six in /usr/local/lib/python3.11/dist-packages (from catboost) (1.17.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.24->catboost) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.24->catboost) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.24->catboost) (2025.1)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (24.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (3.2.1)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.11/dist-packages (from plotly->catboost) (9.0.0)
Downloading catboost-1.2.7-cp311-cp311-manylinux2014_x86_64.whl (98.7 MB)
98.7/98.7 MB 7.5 MB/s eta 0:00:00
Installing collected packages: catboost
Successfully installed catboost-1.2.7
```

Гигантский блок импорта

[Показать код](#)

Набор данных

Набор данных представляет собой статистику параметров автомобилей на вторичном рынке в Молдавии.

Набор включает ряд категориальных и численных значений, составляющих одну запись (строку). Число записей можно найти как число строк.

Каждый столбец в записи — это отдельный параметр.

Среди указанных параметров приведены целевой для задачи предсказания (регрессии) - цена автомобиля.

Также среди параметров есть целевой для задачи классификации - тип трансмиссии.

Последняя задача может быть рассмотрена, например, как пример задачи на заполнение пропусков (если продавец не указал соответствующий параметр).

Загрузка данных

Загружаем наши данные из файла по прямой ссылке на git-hub

```
DF = pd.read_csv('https://raw.githubusercontent.com/dayekb/Basic_ML_Alg/main/cars_moldova_clean.csv', delimiter = ',')
DF
```



	Make	Model	Year	Style	Distance	Engine_capacity(cm3)	Fuel_type	Transmission	Price(euro)
0	Toyota	Prius	2011	Hatchback	195000.0	1800.0	Hybrid	Automatic	7750.0
1	Renault	Grand Scenic	2014	Universal	135000.0	1500.0	Diesel	Manual	8550.0
2	Renault	Laguna	2012	Universal	110000.0	1500.0	Diesel	Manual	6550.0
3	Opel	Astra	2006	Universal	200000.0	1600.0	Metan/Propan	Manual	4100.0
4	Mercedes	Vito	2000	Microvan	300000.0	2200.0	Diesel	Manual	3490.0
...	...	...	...	...	...	...	...	...	...
32480	Volkswagen	Passat	2016	Sedan	88000.0	1800.0	Petrol	Automatic	11500.0
32481	Land Rover	Freelander	2002	Crossover	225000.0	1800.0	Metan/Propan	Manual	4400.0
32482	Dacia	Logan Mcv	2015	Universal	89000.0	1500.0	Diesel	Manual	7000.0
32483	Mazda	6	2006	Combi	370000.0	2000.0	Diesel	Manual	4000.0
32484	Renault	Grand Scenic	2006	Minivan	300000.0	1500.0	Diesel	Manual	4000.0

32485 rows × 9 columns

```
# Добавим пару случайных признаков для последующего анализа
DF['random_feature_1'] = np.random.random(size = DF.shape[0]) # равномерное распределение
DF['random_feature_2'] = np.random.normal(size = DF.shape[0]) # нормальное распределение
```

Список числовых и категориальных данных

```
cat_columns = ['Make', 'Model', 'Style', 'Fuel_type', 'Transmission']
num_columns = ['Year', 'Distance', 'Engine_capacity(cm3)', 'Price(euro)']
```

Предварительная обработка из [предыдущего блокнота](#)

Из предварительной обработки исключена колонка `Price(euro)`, поскольку она является целевой меткой и будет обработана отдельно

> Всякие там Pipeline

[Показать код](#)

> Вспомогательные функции

[Показать код](#)

```
scoring_reg = {'R2': 'r2',
               '-MSE': 'neg_mean_squared_error',
               '-MAE': 'neg_mean_absolute_error',
               '-Max': 'max_error'}

cv_rule = ShuffleSplit(n_splits=5, random_state = 42)

metrics = [mse, r2_score]

metric_names = ['mse', 'r2_score']
```

✓ Линейная Регрессия

Считываем данные, разбиваем на тестовую и тренировочную

```
# не забываем удалить целевую переменную цену из признаков
X,y = DF.drop(columns = ['Price(euro)']), DF['Price(euro)']

# разбиваем на тестовую и валидационную
X_train, X_val, y_train, y_val = train_test_split(X, y,
                                                  test_size=0.3,
                                                  random_state=42)
```

Запускаем Pipeline из предварительной обработки и модели

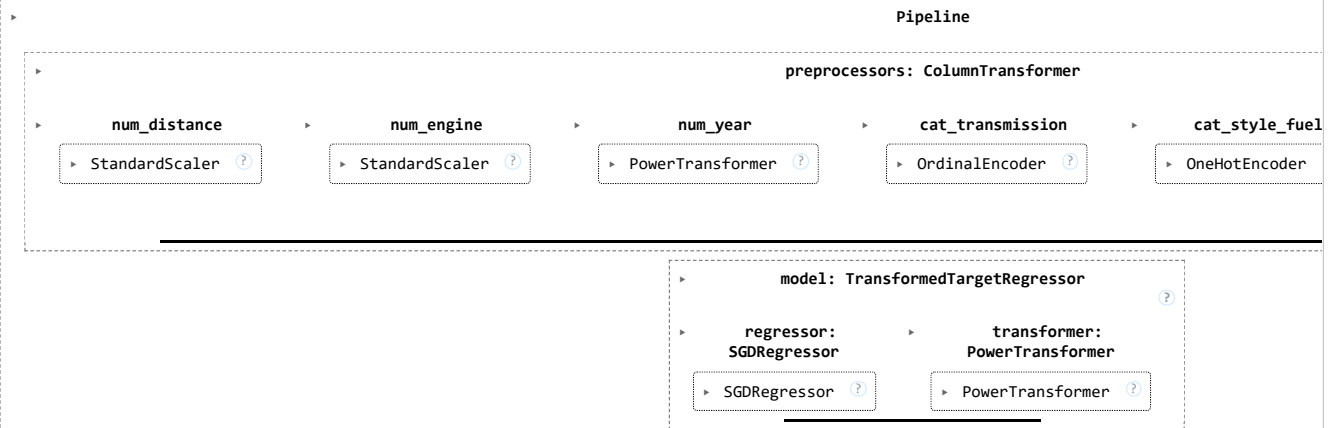
```
pipe_all_transform = Pipeline([
    ('preprocessors', preprocessors_all),
    ('model', TransformedTargetRegressor( regressor=SGDRegressor(random_state = 42),
                                     transformer=PowerTransformer()))
```



```
)
])
```

```
%time
pipe_all_transform.fit(X_train, y_train)
```

```
CPU times: user 132 ms, sys: 3.04 ms, total: 135 ms
Wall time: 141 ms
```



✓ CatBoost

```
train_data = Pool(
    data=X_train,
    label=y_train,
    cat_features = cat_columns,
    feature_names=list(X_train.columns)
)

eval_data = Pool(
    data=X_val,
    label=y_val,
    cat_features = cat_columns,
    feature_names=list(X_train.columns)
)
```

```
model_reg = CatBoostRegressor(iterations = 1000,
    early_stopping_rounds=100,
    verbose = 100,
    cat_features = cat_columns,
    depth = 3,
    eval_metric= 'MAE'
)
```

```
%time
model_reg.fit(X=train_data,
    eval_set=eval_data,
    );
```

```
Learning rate set to 0.083159
0:   learn: 5341.6428495   test: 5427.6644559   best: 5427.6644559 (0) total: 58.3ms   remaining: 58.2s
100: learn: 2049.4252973   test: 2073.1290595   best: 2073.1290595 (100) total: 1.14s   remaining: 10.1s
200: learn: 1921.3654333   test: 1951.8083915   best: 1951.8083915 (200) total: 2.74s   remaining: 10.9s
300: learn: 1869.9018151   test: 1905.7687038   best: 1905.7456483 (298) total: 4.18s   remaining: 9.7s
400: learn: 1837.8362692   test: 1879.8009912   best: 1879.8009912 (400) total: 6.4s   remaining: 9.56s
500: learn: 1810.3576702   test: 1857.5614446   best: 1857.5614446 (500) total: 9.02s   remaining: 8.99s
600: learn: 1788.8341804   test: 1844.9043402   best: 1844.6543820 (593) total: 11.7s   remaining: 7.75s
700: learn: 1767.3838201   test: 1830.2629852   best: 1830.2629852 (700) total: 14.7s   remaining: 6.25s
800: learn: 1753.5863422   test: 1822.2390824   best: 1822.2390824 (800) total: 16.3s   remaining: 4.05s
900: learn: 1740.5654641   test: 1812.6525217   best: 1812.6525217 (900) total: 19.5s   remaining: 2.14s
999: learn: 1727.0407682   test: 1804.4188520   best: 1804.4188520 (999) total: 21.5s   remaining: 0us
```

```
bestTest = 1804.418852
bestIteration = 999
```

```
CPU times: user 12.9 s, sys: 1.19 s, total: 14.1 s
Wall time: 21.9 s
<catboost.core.CatBoostRegressor at 0x7d176d9ee6d0>
```

✓ Shap



Надо устанавливать библиотеку Shap - но тут стандартно

```
!pip install shap
```

```
Requirement already satisfied: shap in /usr/local/lib/python3.11/dist-packages (0.46.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from shap) (1.26.4)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from shap) (1.13.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.11/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.11/dist-packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.11/dist-packages (from shap) (24.2)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.11/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba in /usr/local/lib/python3.11/dist-packages (from shap) (0.61.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.11/dist-packages (from shap) (3.1.1)
Requirement already satisfied: llvmlite<0.45,>=0.44.0dev0 in /usr/local/lib/python3.11/dist-packages (from numba->shap) (0.44.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas->shap) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas->shap) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas->shap) (2025.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn->shap) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn->shap) (3.5.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
```

```
import shap
shap.initjs()
```



✖ Для линейной модели

✖ Первая боль

С `pipeline` пока полноценную интеграцию не завезли

Поэтому трансформируем исходные данные, обратившись к части обученного `pipeline`

```
data_transform = pipe_all_transform['preprocessors'].transform(X_val)
```

```
feature_names = [x.split('__')[1] for x in pipe_all_transform['preprocessors'].get_feature_names_out()]
```

✖ Интерфейс работы в shap

Создаем объект класса

И затем его применяем к данным

Наиболее общий - `Explainer`

```
%time
# нужно задать метод predict от объекта класса модели машинного обучения. любой
explainer_line = shap.Explainer(pipe_all_transform['model'].regressor_.predict, # модель (не pipeline)
                                data_transform, # данные (которые подставляются в итоговую модель)
                                feature_names=feature_names
                                # название признаков. чтобы не скучно "feature 1"
                                )

#
shap_values_line = explainer_line(data_transform)

CPU times: user 2 µs, sys: 0 ns, total: 2 µs
Wall time: 6.44 µs
PermutationExplainer explainer: 9747it [01:58, 73.64it/s]
```

Полученный объект - своеобразный словарь где всё

```
shap_values_line
```

```
.values =
array([[ 5.74289802e-03,  5.49346797e-01,  2.13657037e-01, ...,
         7.76391055e-01,  2.64689240e-04, -5.52205880e-03],
       [ 2.86387333e-03,  5.92103660e-02,  2.13657037e-01, ...,
         1.66432091e-01, -1.52279299e-04, -4.28249771e-04],
       [ 3.28323975e-05, -9.16008435e-02,  1.20481390e-01, ...,
         8.71180383e-02,  2.27018118e-05, -7.04269155e-03],
       ...,
       [ 3.43967827e-03,  8.97496359e-02,  2.94015688e-02, ...,
        -3.89928730e-02,  3.05137995e-04,  1.21909784e-03],
       [-5.48529824e-03,  4.73941192e-01, -3.96140521e-01, ...]
```



```

4.22832438e-01, 1.41044346e-05, -4.58147651e-03],
[ 3.39169453e-03, -5.38980411e-02, 4.06485441e-01, ...,
 3.50295798e-02, -9.60028138e-05, 3.28196758e-03]])

.base_values =
array([-0.01389082, -0.01389082, -0.01389082, ..., -0.01389082,
      -0.01389082, -0.01389082])

.data =
array([[ -1.45274682,  4.68549151,  0.33699568, ...,  3.25088106,
         0.14043045,  1.02753136],
       [-0.79890429,  0.57577082,  0.33699568, ...,  0.62085429,
         0.81245795, -0.04545089],
       [-0.15595915, -0.68875863,  0.18254817, ...,  0.27886722,
         0.53044119,  1.3478441 ],
       ...,
       [-0.9296728 ,  0.83183803,  0.03157469, ..., -0.26489899,
         0.07523926, -0.39245541],
       [ 1.09723902,  4.05322679, -0.67380187, ...,  1.7264037 ,
         0.54429756,  0.82940298],
       [-0.91877542, -0.37262627,  0.65662713, ...,  0.05427171,
         0.72175723, -0.82698732]])

```

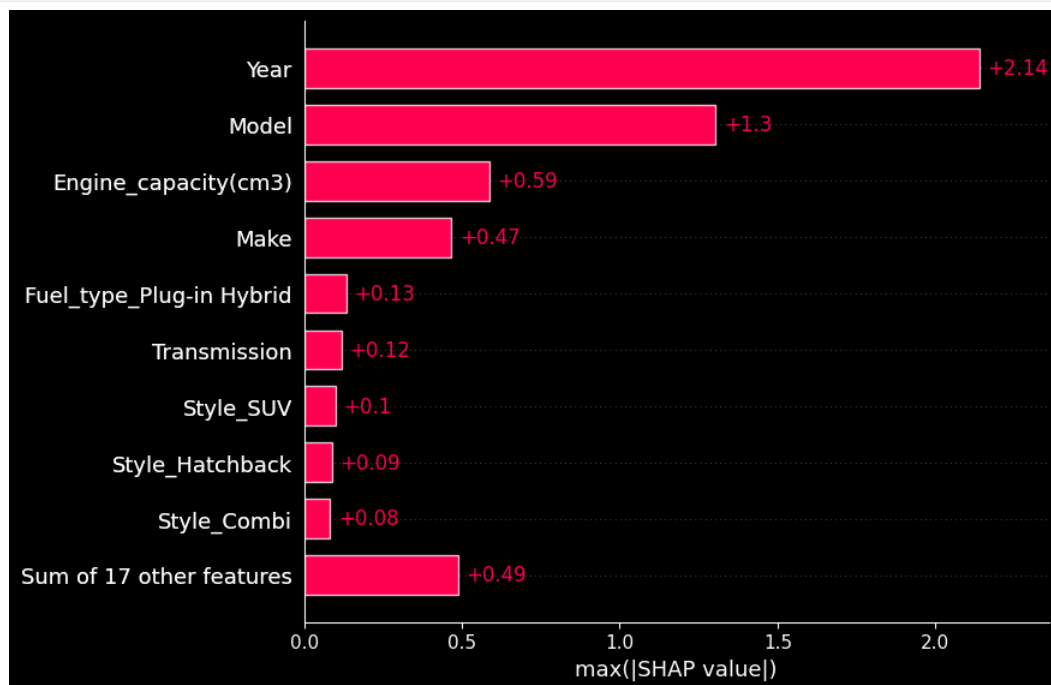
Графиков тут много

простые столбчатые диаграммы

Для глобальной интерпретируемости

Почти как feature / permutation importance

```
shap.plots.bar(shap_values_line.abs.max(0))
```



Year и **Model** всё ещё в топе

Скатерограмма + Что-то от столбчатой диаграммы

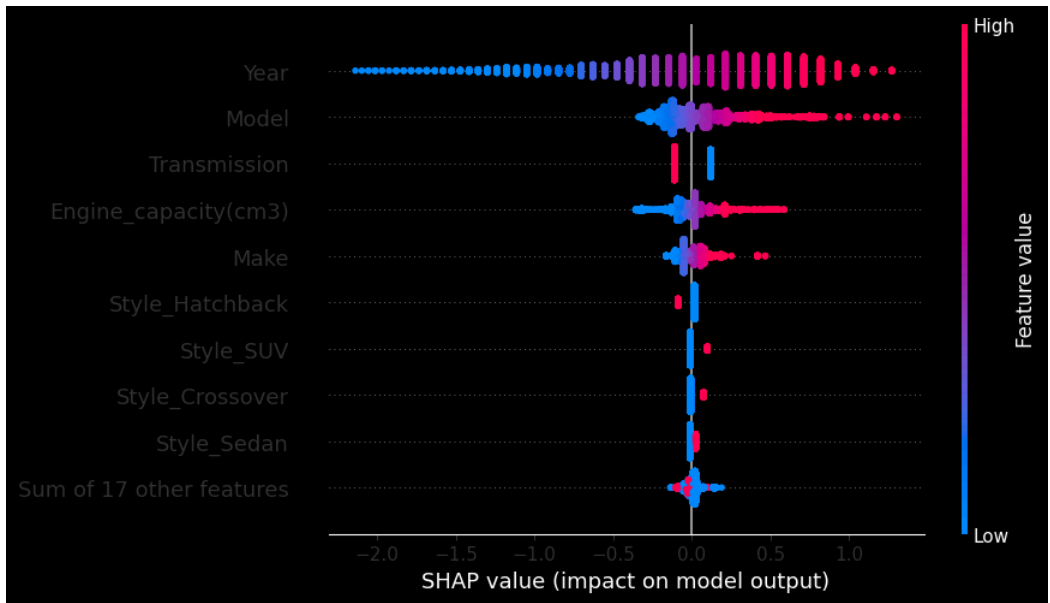
График поинтересней

Тут отсортированы по "разросу влияния" (по вертикали)

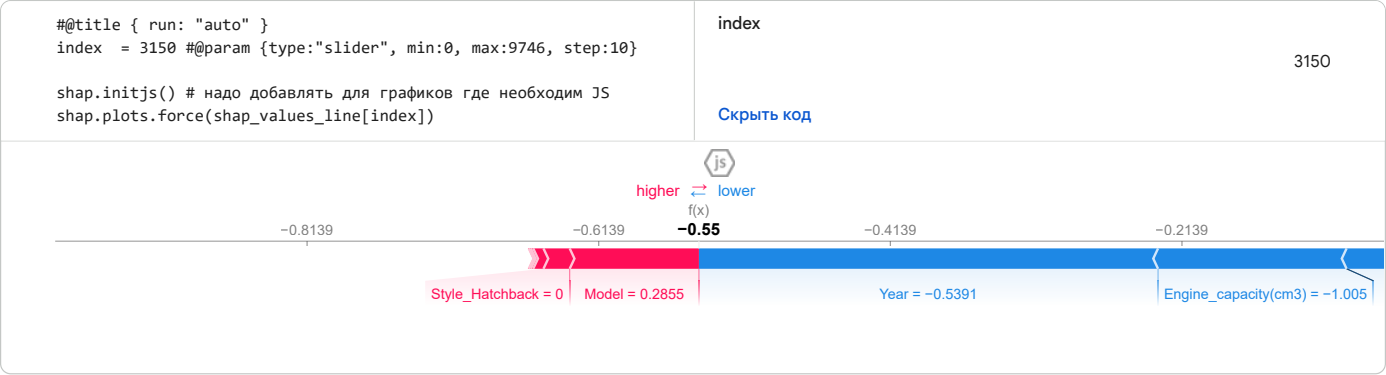
По горизонтали - как конкретное значение признака влияет на предсказание

Цветом - значение признака (от минимального голубого до максимального красного)

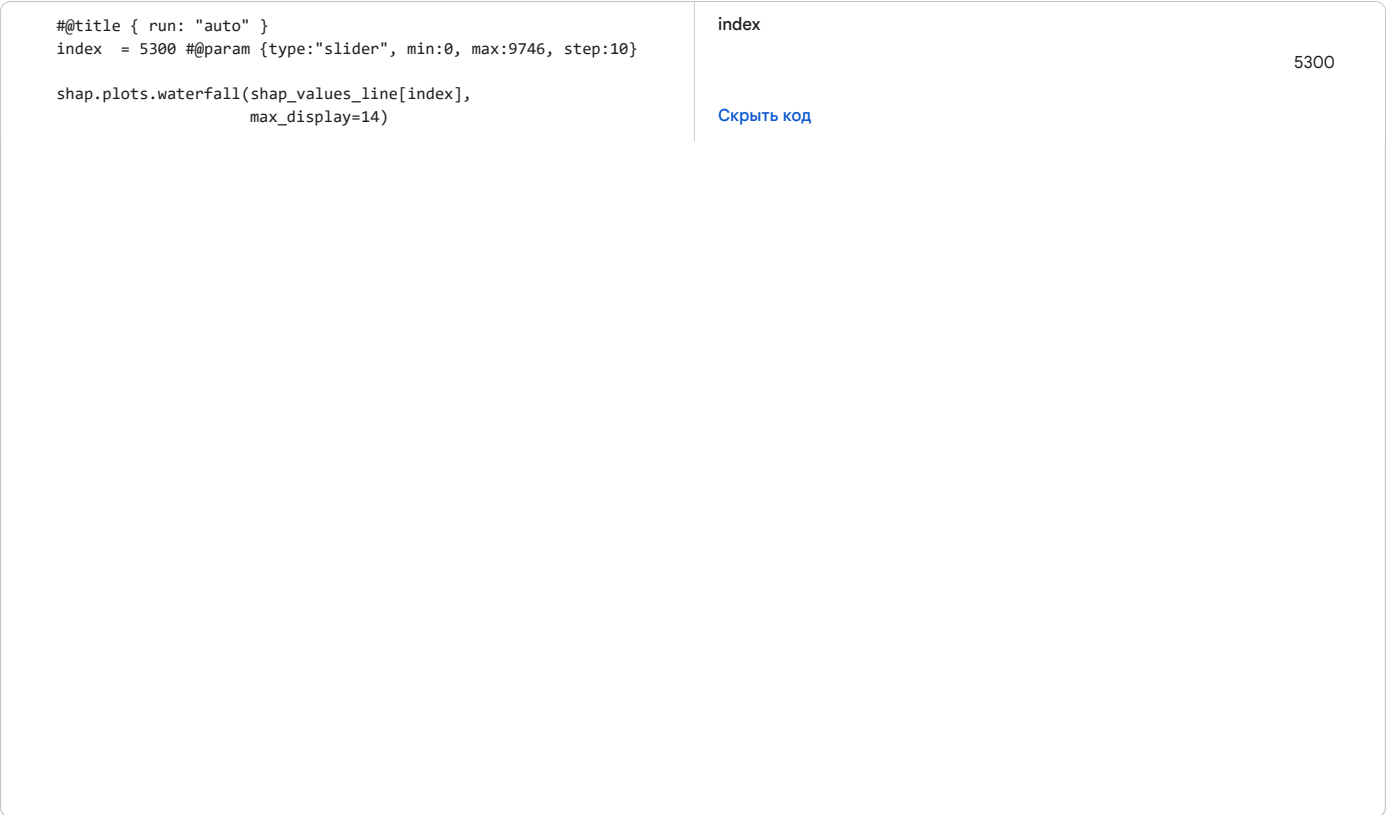
```
shap.plots.beeswarm(shap_values_line)
```

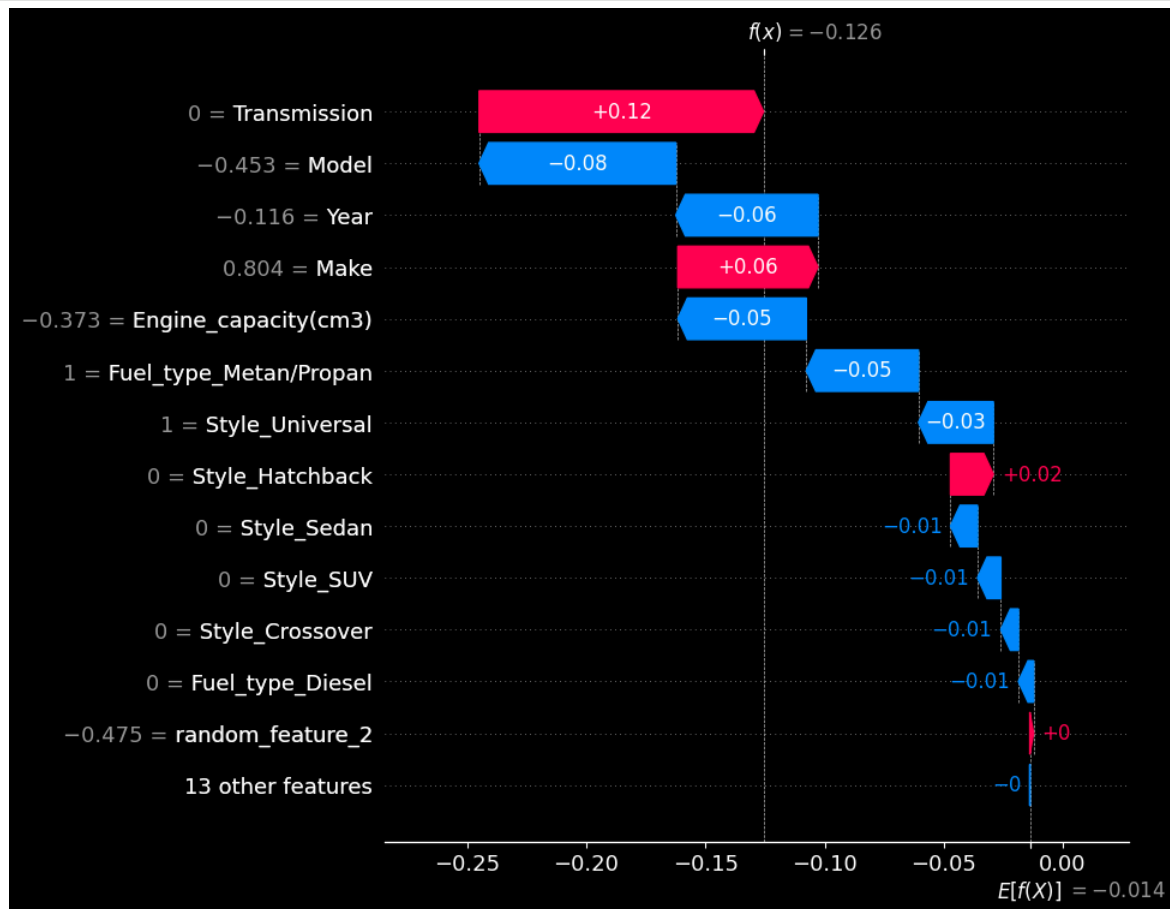


Для индивидуальных объяснений



Еще один индивидуальный график





Несколько индивидуальных предсказаний

```
##@title { run: "auto" }
index = 2900 #@param {type:"slider", min:0, max:9646, step:10}

f1 = plt.figure()

shap.plots.heatmap(shap_values_line[index:index+100],
                  );

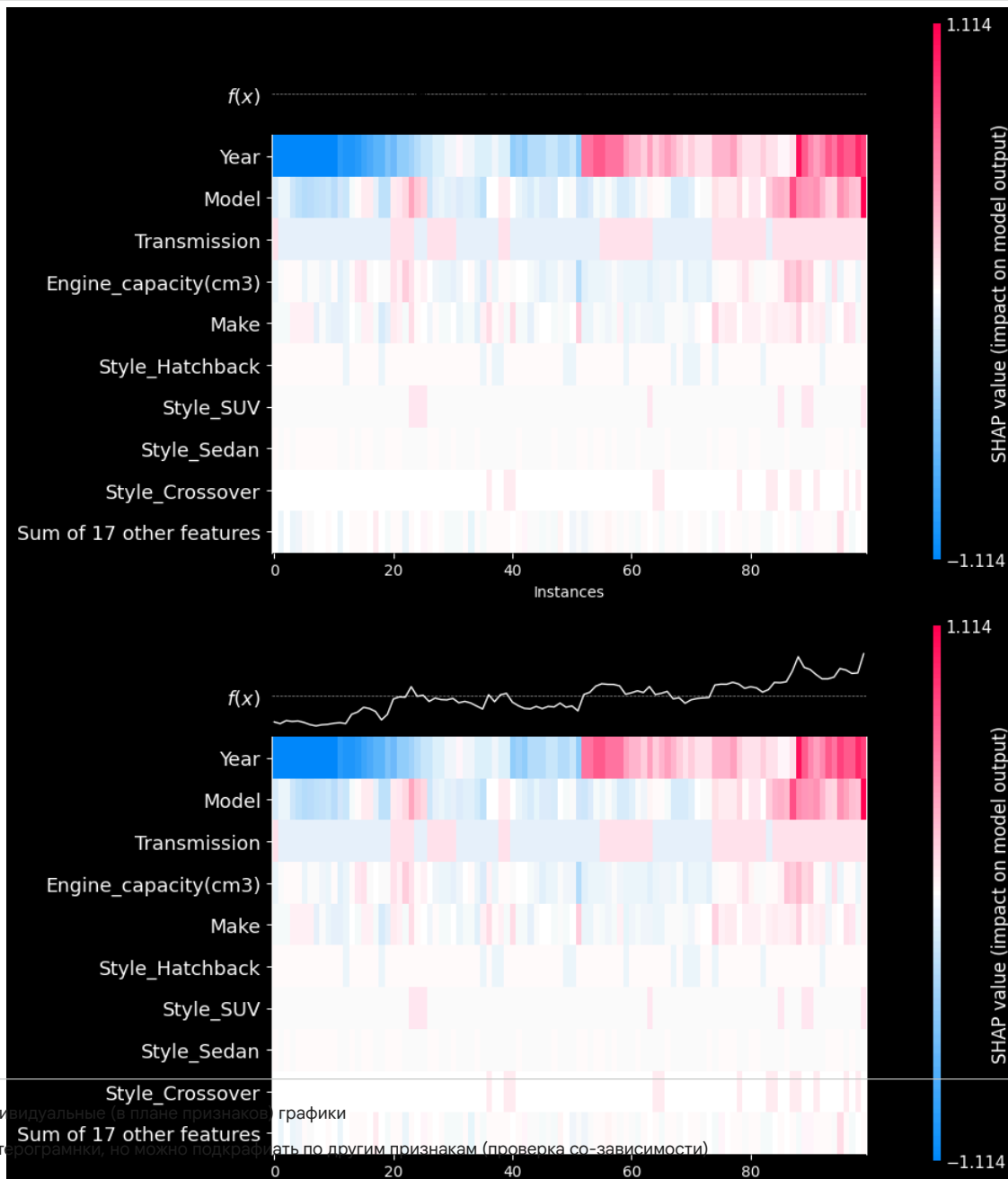
# за темный фон, как за любые принципы нужно страдать
f1.gca().get_lines()[1].set_color('white')

f1
```

index

2900

[Скрыть код](#)



Индивидуальные (в плане признаков) графики

Скатерограммы, но можно подкрепить по другим признакам (проверка со-зависимости)

shap_values_line

```
.values =
array([[ 5.74289802e-03,  5.49346797e-01,  2.13657037e-01, ...,
        7.76391055e-01,  2.64689240e-04, -5.52205880e-03],
       [ 2.86387333e-03,  5.92103660e-02,  2.13657037e-01, ...,
        1.66432091e-01, -1.52279299e-04, -4.28249771e-04],
       [ 3.28323975e-05, -9.16008435e-02,  1.20481390e-01, ...,
        8.71180383e-02,  2.27018118e-05, -7.04269155e-03],
       ...,
       [ 3.43967827e-03,  8.97496359e-02,  2.94015688e-02, ...,
       -3.89928730e-02,  3.05137995e-04,  1.21909784e-03],
       [-5.48529824e-03,  4.73941192e-01, -3.96140521e-01, ...,
        4.22832438e-01,  1.41044346e-05, -4.58147651e-03],
       [ 3.39169453e-03, -5.38980411e-02,  4.06485441e-01, ...,
        3.50295798e-02, -9.60028138e-05,  3.28196758e-03]])

.base_values =
array([[-0.01389082, -0.01389082, -0.01389082, ..., -0.01389082,
       -0.01389082, -0.01389082])

.data =
array([[ -1.45274682,  4.68549151,  0.33699568, ...,  3.25088106,
         0.14043045,  1.02753136],
       [-0.79890429,  0.57577082,  0.33699568, ...,  0.62085429,
         0.81245795, -0.04545089],
       [-0.15595915, -0.68875863,  0.18254817, ...,  0.27886722,
         0.53044119,  1.3478441 ],
       ...,
       [-0.9296728 ,  0.83183803,  0.03157469, ..., -0.26489899,
```




```
0.07523926, -0.39245541],
[ 1.09723902,  4.05322679, -0.67380187, ...,  1.7264037 ,
 0.54429756,  0.82940298],
[-0.91877542, -0.37262627,  0.65662713, ...,  0.05427171,
 0.72175723, -0.82698732]])
```

```
##@title { run: "auto" }
index_plot = "Year" #@param ['Distance', 'Engine_capacity(cm3)', '
index_color = "Year" #@param ['Distance', 'Engine_capacity(cm3)',

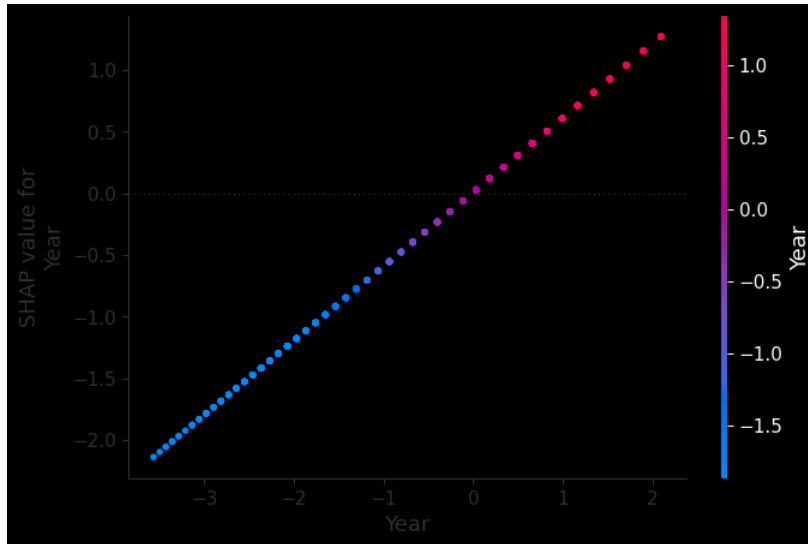
shap.plots.scatter(
    shap_values_line[:,index_plot],
    color=shap_values_line[:,index_color]
)
```

index_plot

Year

index_color

Year

[Скрыть код](#)

▼ Для CatBoost

```
%time
explainer_cb = shap.TreeExplainer(model_reg,
                                   feature_names=X_val.columns)
```

```
shap_values_cb = explainer_cb(X_val)
```

```
CPU times: user 3 µs, sys: 0 ns, total: 3 µs
Wall time: 6.91 µs
```

```
shap.plots.bar(shap_values_cb.abs.max(0))
```

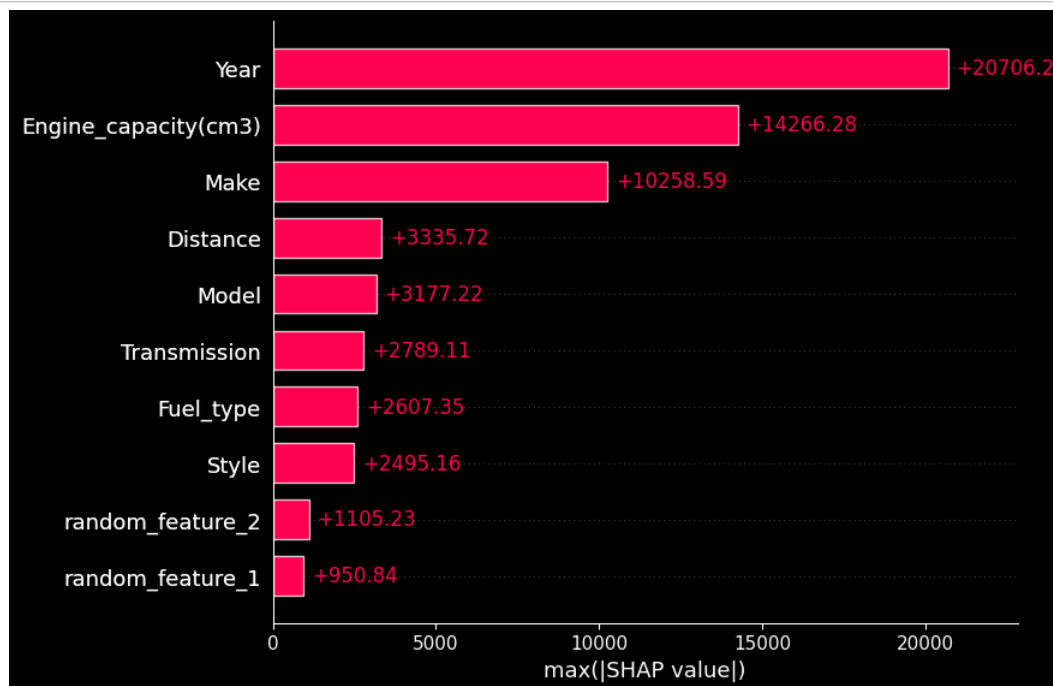
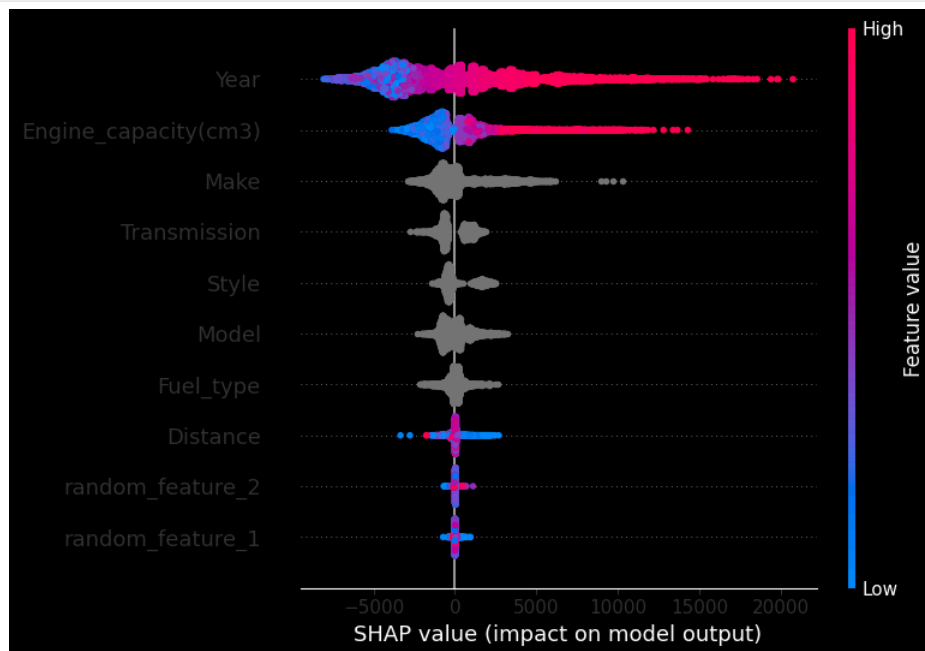


График поинтересней

```
shap.summary_plot(shap_values_cb, X_val, )
```



Для индивидуальных объяснений

```
shap_values_cb
```

```
.values =
array([[ 3792.90912973,  1941.76453349,  1104.12421699, ...,
         587.57262711,  -45.14665426,  162.39190376],
       [-173.01877966,  793.42767281, -154.04516991, ...,
         855.79990267,  27.24330247, -111.37901505],
       [ 1614.78052633, -74.26361689, -950.06899092, ...,
         642.98711763,  16.71808353,  11.08903446],
       ...,
       [-423.40450674, -208.99992845, -1936.589474, ...,
         709.8257603,   -7.68531579,   25.6754492 ],
       [ 1316.03813431,  262.99440011, -6763.61779485, ...,
         489.73970646, -22.39304293, -191.70670489],
       [-1027.373731,  163.57112814, 1445.61252357, ...,
         820.11812918,  30.13868692,  25.23209591]])
```



```
.base_values =
array([[8928.46012226, 8928.46012226, 8928.46012226, ..., 8928.46012226,
      8928.46012226, 8928.46012226]])

.data =
array([[ 'Porsche', 'Cayenne', 2011, ..., 'Automatic',
      0.14043045036113477, 1.0275313573101332],
 [ 'Honda', 'CR-V', 2011, ..., 'Automatic', 0.8124579513697127,
      -0.04545088986105271],
 [ 'Audi', 'A3', 2010, ..., 'Automatic', 0.5304411861551458,
      1.3478440990731044],
 ...,
 [ 'Honda', 'Accord', 2009, ..., 'Automatic', 0.07523925963866274,
      -0.39245540684114283],
 [ 'BMW', 'X5', 2004, ..., 'Automatic', 0.5442975647511861,
      0.829402983295719],
 [ 'Nissan', 'Juke', 2013, ..., 'Automatic', 0.7217572335789229,
      -0.8269873246800378]], dtype=object)
```

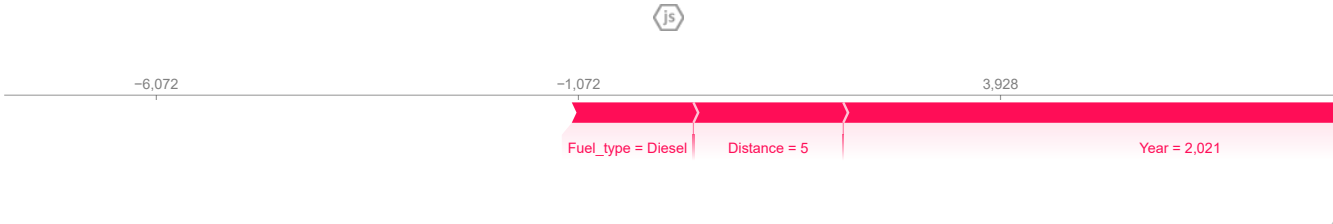
```
##@title { run: "auto" }
index = 4030 ##@param {type:"slider", min:0, max:9746, step:10}

shap.initjs() # надо добавлять для графиков где необходим JS
shap.plots.force(shap_values_cb[index], X_val)
```

index

4030

Скрыть код



Еще один индивидуальный график

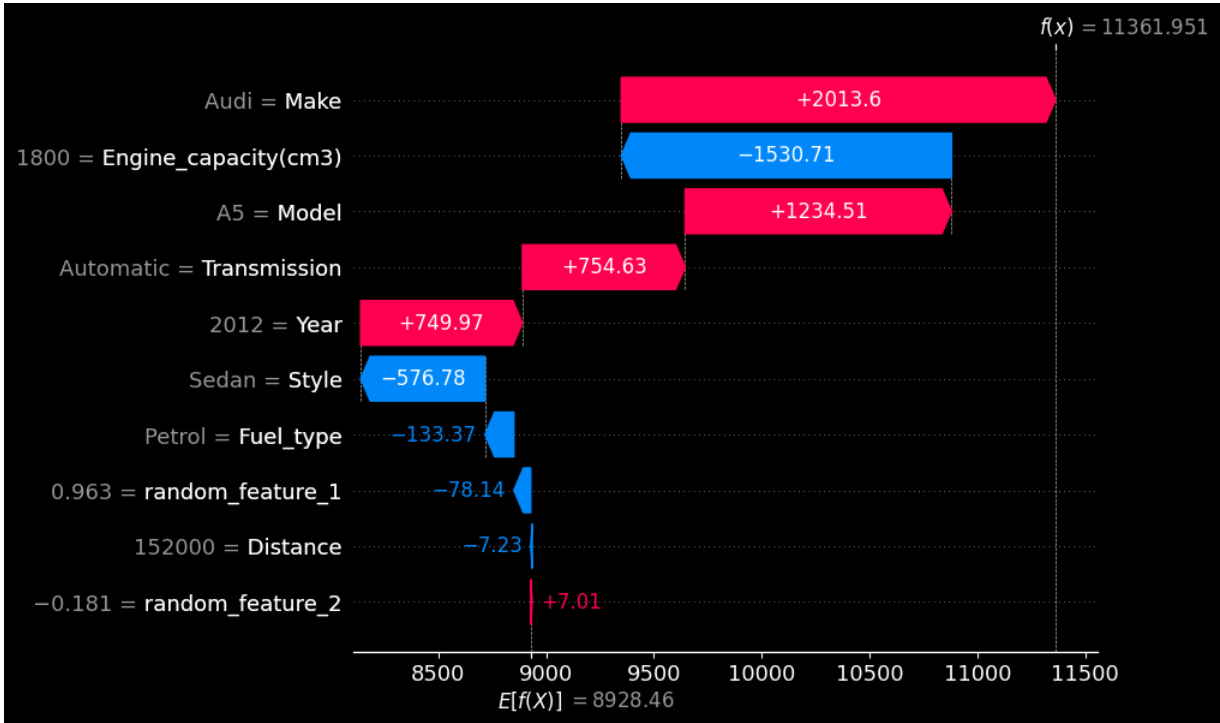
```
##@title { run: "auto" }
index = 2620 ##@param {type:"slider", min:0, max:9746, step:10}

shap.plots.waterfall(shap_values_cb[index],
                    max_display=14)
```

index

2620

Скрыть код



Несколько индивидуальных предсказаний

```
##@title { run: "auto" }
index = 2900 ##@param {type:"slider", min:0, max:9646, step:10}

f1 = plt.figure()
```

index

2900

Скрыть код



```
shap.plots.heatmap(shap_values_cb[index:index+100])  
  
# за темный фон, как за любые принципы нужно страдать  
f1.gca().get_lines()[1].set_color('white')  
  
f1
```

